◆ DRAGONAI · 烛龙智元

AI FDE 工程师就业实训营

学生讲义

Day 4

从 LLM 到 Agent 与 RAG

从函数讲到大语言模型，再讲到 RAG 与智能体

杭州烛龙智元科技有限公司

目录

学习目标 1

01 开场与地图 1

1.1 热词与地图 1

1.2 人工智能的定义 1

1.3 主线与学习路径 2

02 函数与 AI 2

2.1 函数与现实符号化 2

2.2 符号主义及其边界 3

2.3 从样本反推规律 3

2.4 联结主义路线 3

2.5 非线性与激活函数 4

2.6 多层组合与逼近 4

2.7 神经网络图与隐藏层 5

03 训练与神经网络 5

3.1 训练与参数 5

3.2 误差与损失函数 6

3.3 参数优化 6

3.4 过拟合与泛化 6

3.5 从数据中学习函数 7

3.6 机器学习与传统编程 8

3.7 深度学习与网络结构 8

04 语言数字化 9

4.1 把语言写成预测问题 9

4.2 文字的编码问题 9

4.3 编号与 One-hot 编码 9

4.4 词向量 9

4.5 余弦相似度与语义空间 10

4.6 嵌入矩阵与序列模型 10

05 大语言模型 11

5.1 LLM 与参数规模 11

5.2 下一词预测与概率 11

5.3 Transformer 与注意力 11

5.4 预训练与对齐 12

5.5 规模与训练成本 12

5.6 闭源模型与开源模型 12

5.7 按场景选模型 13

06 微调与部署 13

6.1 从通才到专才 13

6.2 毛坯房与精装修 14

6.3 微调的常见类型 14

6.4 微调的投入与归属 14
6.5 本地部署的价值 14
6.6 硬件与显存估算 15
6.7 本地部署的时机 15
07 RAG 工程 16
7.1 幻觉与检索 16
7.2 检索增强生成 16
7.3 入库与查询 16
7.4 RAG 解决的问题 17
7.5 标准流程与向量数据库 17
7.6 检索环节的优化 17
7.7 Chunk 与切块策略 18
7.8 文件处理与 Markdown 18
7.9 docx 与 PDF 的处理 19
7.10 OCR 识别与文件入库 19
08 Agent 与总结 20
8.1 智能体的组成与分类 20
8.2 智能体开发框架 20
8.3 agno 智能体的结构 20
8.4 从函数到智能体 21
练习与作业 22

复习题 22

课后作业 22

学习目标

第四天沿着从函数到大语言模型的主线，先理解模型在学什么，再理解 RAG 与智能体怎样把模型能力接到真实数据与真实工具上。

1. 说明符号主义为什么会遇到边界，联结主义怎样用数据逼近规律，以及激活函数与多层结构的作用。
2. 说明训练神经网络就是寻找参数的过程，解释损失函数与优化的作用，并区分过拟合与泛化。
3. 区分每个词一个编号，One-hot 编码与词向量这几种编码方式，说明余弦相似度与嵌入矩阵的作用。
4. 说明大语言模型以预测下一个词为核心的工作方式，区分预训练，对齐与微调，并能根据效果与成本，以及数据隐私与部署条件，在 API 调用与开源部署之间做出选择。
5. 说明 RAG 准备资料与回答问题两个阶段的流程，以及 OCR 怎样把图片型 PDF 变成可检索的文本。
6. 说明智能体在大模型之外还需要工具，记忆与规划，以及用 agno 搭建带资料回答的智能体的基本结构。

01
开场与地图

1.1 热词与地图

AI 领域常被同时提起的热词有八个：

Codex	Claude Code	LLM	RAG
Agent	具身智能	token	openclaw

这些词分属工具，模型与具身智能等不同层面，初学者容易被名词吓住。理解这些词，办法是先建立一张能听懂 AI 的地图，再把每个词放回底层逻辑里。底层逻辑清楚了，每个词在地图上的位置也就清楚了。

1.2 人工智能的定义

人工智能可以先用一个简明的定义来理解：**人工智能**是让机器完成原本需要人来完成的智能任务。这类任务按顺序分为四个环节，前一个环节的结果交给后一个环节：

1. 感知：看见世界。
2. 理解：理解含义。

3. 生成：组织表达。

4. 行动：调用工具完成操作。

1.3 主线与学习路径

AI 是一个很大的体系。数学方面涉及微积分与概率论，以及线性代数与“最优化”。技术方面涉及机器学习与深度学习，以及数据处理与工程系统。这些学科不必一次学完，第四天只抓一条主线：LLM 与 RAG。主线上的概念按这个顺序展开：

1. 函数：用 $y = f(x)$ 描述输入与输出之间的规则。

2. 机器学习：从数据中学习函数。

3. 神经网络：用大量简单单元连接成网络，逼近复杂的函数。

4. 词向量：把词变成可以计算的向量。

5. LLM：以语言预测为核心的大型神经网络。

6. RAG：先检索资料，再生成答案。

这个顺序先回答“模型在学什么”，再回答“为什么 RAG 能落地”。每个概念都以前一个概念为基础。

小结

理解 AI 热词的办法是先有一张地图，再把每个词放回底层逻辑。人工智能指机器完成原本需要人来完成的智能任务，过程由感知到行动逐步推进。第四天只沿 LLM 与 RAG 这条主线学习，以函数为起点，经过机器学习，神经网络与词向量，再到大语言模型与 RAG。

02
函数与 AI

2.1 函数与现实符号化

AI 的底层原理从函数讲起。函数写作 $y = f(x)$：输入 x 经过规则 f，得到输出 y。三者有先后顺序，先有输入，再按规则处理，然后得到输出。这个思想可以概括为“function describes the world”，即函数描述世界：给定一个输入，按某种规则得到一个输出，很多现象都可以这样描述。

AI 要处理现实问题，先要把现实世界符号化：把现实中的事物写成符号，作为输入 x，由规则 f 处理，再用得到的输出 y 解释世界。AI 的第一步，就是把问题变成这种可计算的形式。问题写成 $y = f(x)$ 之后，要解决的就是找到规则 f。

2.2 符号主义及其边界

早期人工智能寻找规则的办法，是由人想清楚规则，再写给机器执行。这条路线叫**符号主义**，英文是 Symbolic AI。符号主义系统接收一个现象作为输入，用规则库中 `IF...THEN...` 形式的规则做判断，输出结论。

符号主义的局限在于，很多规则人类自己也写不清楚。以识别小狗为例，耳朵，尾巴和毛色都因狗而异，想用规则描述小狗，规则会越来越多，越来越零散，仍然覆盖不全。翻译也一样，词典能给出词与词的对应，但语境和歧义很难用固定的规则写全。

既然写不出这个函数，就改变思路：不再手工编写规则，而是从样本中推出规律，让模型从数据中学习。现代机器学习就建立在这个思路上。

2.3 从样本反推规律

已知几组 x 和 y，可以反推出两者之间的规律。如果每个 y 都是 x 的两倍，样本点就排在一条直线上，规律是 $y = 2x$。规律足够简单时，少量样本就能看出来。

规律看不出来时，常用的办法是先假设一个函数，再看这个函数的误差有多大。形象地说，就是先“猜”一条直线 $y = wx + b$。准确地说，这一步是给参数 w 和 b 取一组初始值，再根据误差逐步调整。w 决定直线的斜率，b 决定直线的上下位置，参数一变，直线的位置和斜率就跟着变。

图中的横轴是 x，纵轴是 y，实心点与空心点都表示已知的样本点。左图是参数取初始值时的直线，样本点逐渐升高，直线却几乎是平的，误差很大。右图是调整参数之后的直线，贴近了大部分样本点。

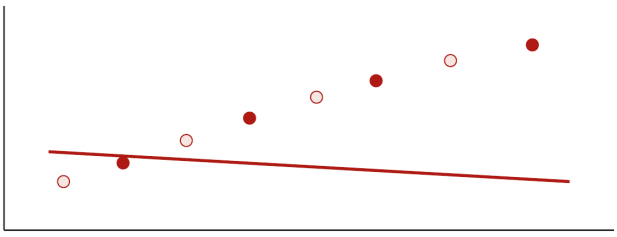


图 1　参数取初始值时，直线几乎是平的，离样本点很远。

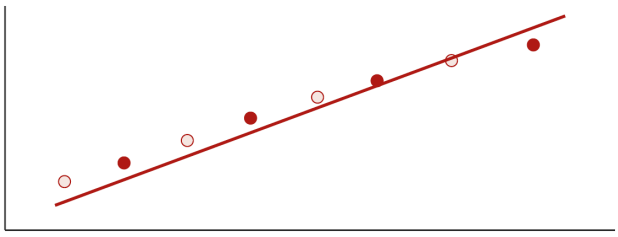


图 2　调整参数之后，直线贴近大部分样本点。

现实数据很少能被一个完美的函数完全描述，数据里往往带有误差。拟合指用函数去贴近样本点，目标是找到足够接近真实数据的近似关系，并不要求函数穿过每一个点。右图的直线没有穿过每一个点，已经是一个可用的近似。

2.4 联结主义路线

先给参数取初始值，再用数据逐步调整，这是现代 AI 的基本思路：用数据逼近规律。找不到完美的公式，就训练一个足够好的近似函数。神经网络与大模型沿用的也是这个思路。

这条路线叫**联结主义**，英文是 Connectionism。联结主义用大量简单单元连接成网络，通过学习调整单元之间的连接强度。规则不再由人手工编写，模型从数据中学到一组参数，规律就体现在这些参数里。神经网络是联结主义的典型代表。

这条路线的有效性已经被反复验证。手写数字识别是经典例子：参数不多的神经网络，也能学会区分 0 到 9 这十个数字的写法。网页 `https://adamharley.com/nn_vis/cnn/3d.html` 用三维图形展示了这样一个识别手写数字的网络。随着这类结果不断出现，联结主义才受到广泛重视。

2.5 非线性与激活函数

$y = wx + b$ 描述的是线性关系，图像是一条直线。数据一复杂，直线就很难贴近真实分布。图中的样本点高低交错，两段直线各自只穿过一个样本点，其余的点都偏离直线。

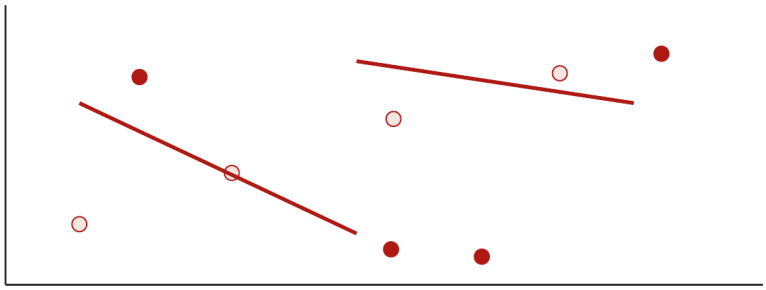


图 3　样本点高低交错，两段直线各自只穿过一个样本点。

让函数变成曲线的办法，是在线性函数外面再套一层非线性函数，这层函数叫**激活函数**，写成 $f(x) = g(wx + b)$。其中 $wx + b$ 是线性部分，g 是激活函数。线性部分的结果经过 g 之后变成非线性输出，形象地说，直线“弯起来”了。常见的激活函数有五种：

激活函数	形状与作用
ReLU	输入小于等于 0 时输出 0，输入大于 0 时原样输出
Sigmoid	S 形曲线，输出在 0 到 1 之间
tanh	S 形曲线，输出在负 1 到 1 之间
sin	正弦曲线，输出在负 1 到 1 之间周期起伏
GELU	形状接近 ReLU，在 0 附近平滑过渡

这些函数的名字带着数学色彩，作用却很直观：把直线变成曲线，让模型能够表达更复杂的关系。

2.6 多层组合与逼近

一层激活函数带来的弯曲有限，可以把函数一层层嵌套起来，形象地说就是“套娃”。输入可以有多个，层也可以有很多，例如 $g\left(w_3 \cdot g\left(w_1 x_1 + w_2 x_2 + b_1\right) + b_2\right)$。内层把两个输入 x_1 与 x_2 按参数 w_1，w_2 和 b_1 组合，再经过激活函数 g。外层把内层的结果乘以 w_3，加上 b_2，再经过一次 g。式子同时具备多个输入，多个参数与多层结构，函数的复杂度就这样叠加起来。

足够复杂的网络，可以逼近很复杂的函数，这就是“万能逼近”的直觉。很多简单变换叠加起来，就能描述复杂的现象，这种表达能力来自函数的组合。

2.7 神经网络图与隐藏层

层层嵌套的式子不便阅读，通常改画成神经网络图。神经网络图中，每个小圆点代表一个神经元，每条连线代表一条连接，每条连接都带有参数。以一个小型网络为例，从左到右分为四层：

层	神经元个数
输入层	3
隐藏层	4
隐藏层	3
输出层	1

相邻两层的神经元两两相连，信息从输入层出发，逐层传到输出层。式子每多嵌套一层 g，网络就多一层神经元。

输入层与输出层之间的层叫隐藏层，英文是 Hidden Layer。这些层夹在中间，从外部看不到层内的计算，所以叫隐藏层。隐藏层把输入逐层变换成更有用的表示。

神经元与层数越多，网络就越复杂。增加隐藏层的神经元，或者增加隐藏层的数量，都会让网络的容量变大，表达能力变强，能够表示更复杂的非线性关系。

小结

AI 处理问题的出发点是把现实写成函数，核心任务是找到其中的规则。手工编写规则的符号主义难以应对识别与翻译这类问题，联结主义转而让模型从数据中学习参数。线性函数只能表示直线，激活函数引入非线性，多层嵌套又让网络能够逼近复杂的函数。神经网络图把这种嵌套画成逐层相连的神经元，中间的隐藏层负责逐层变换输入。

03
训练与神经网络

3.1 训练与参数

神经网络的结构确定之后，模型要学的就是每条连接上的参数 w 和 b。训练神经网络就是在找参数：给定一批样本数据 (x, y)，通过训练得到一组参数 (w, b)。其中样本是已知答案的数据，训练是用样本反复调整参数的过程，参数是训练的结果。已知答案的样本足够多，模型就能逐步调出一组可用的参数。

参数的好坏，看模型的输出与真实答案有多接近。w 和 b 越合适，预测越准确。训练的目标，就是逐步减小预测的误差。

3.2 误差与损失函数

减小误差之前，先要衡量误差的大小。拟合效果的好坏可以直接在图上看出来，依据是预测线与真实点之间的距离。图中的实心点与空心点都是样本点，斜向上的直线穿过样本点，拟合得好，接近水平的直线离大多数样本点很远，拟合得差。图上离得远，在数学上就是误差大。

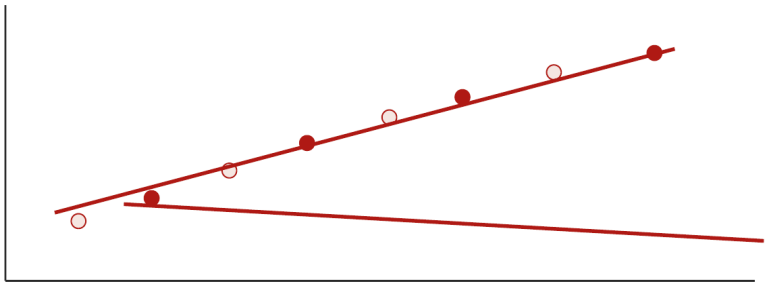


图 4　两条预测线的对比：斜向上的线贴着样本点，接近水平的线远离样本点。

损失函数把“错了多少”变成一个数字。预测线与真实点的距离越小，损失越小。有了这个数字，训练就有了明确的方向：持续让损失变小。

3.3 参数优化

求出让损失最小的参数，是一个优化问题。求解过程分四步：

1. 给参数取一组随机的初始值。
2. 用当前参数计算损失。
3. 根据损失更新参数。
4. 损失变小之后，回到第二步继续，直到损失足够小。

梯度下降是其中一种参数更新方法，做法是沿着让损失下降的方向，把参数每次改动一小步。复杂神经网络的训练，背后是大规模的数学优化。

3.4 过拟合与泛化

网络越大，表达能力越强，但网络越大并不一定越好。模型过于复杂时会出现**过拟合**：模型把训练数据连同其中的误差一起记了下来，形象地说，就是把训练题背了下来。图中的实心点与空心点都是训练用的样本点，曲线为了穿过每一个样本点反复起伏，越过最后一个样本点之后又急剧下降。拟合的目标是近似关系，模型只记住训练数据，就没有学到数据背后的规律。

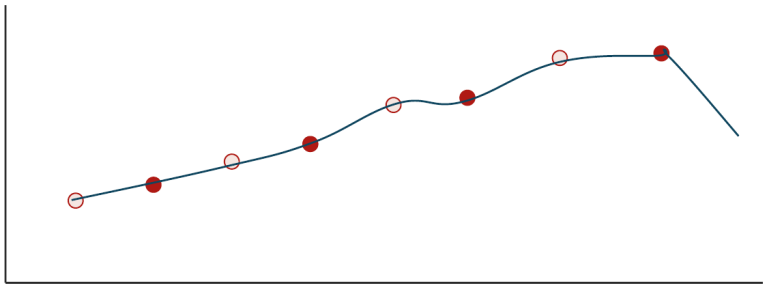


图 5　过拟合的曲线为了穿过每一个样本点反复起伏，越过最后一个点后急剧下降。

与过拟合相对的是**泛化**能力，英文是 Generalization，指模型在没见过的新数据上也能做出准确的预测。训练时使用的数据叫训练集。过拟合的模型在训练集上表现很好，换一批新数据，准确率就明显下降。泛化好的模型在训练集上表现不错，在新数据上同样可用。训练模型追求的是泛化能力。

3.5　从数据中学习函数

参数，损失与优化放在一起，构成了机器学习的完整含义：从数据中学习函数，规则由数据决定。机器学习的使用分为训练与推理两个阶段：

阶段	输入	输出
训练	数据 Data	模型 Model
推理	新数据与模型	预测 Prediction

训练阶段用数据得到模型，推理阶段把新数据交给模型，得到预测。回到 $y = f(x)$，模型就是训练得到的那个函数 f，推理就是用 f 计算新的 x 对应的 y。

机器学习按训练数据有没有答案，分为监督学习，无监督学习与半监督学习，这里的答案就是 y：

学习方式	训练数据	典型任务
监督学习	有 x，也有 y	分类与回归
无监督学习	只有 x	聚类与降维
半监督学习	少量 y，大量 x	分类与回归

半监督学习只需为少量数据标出答案，可以降低标注成本。表中的四种典型任务，含义与例子分别是：

1. 分类：判断数据属于哪一类，例如识别一张手写数字图片是 0 到 9 中的哪一个。
2. 回归：预测一个连续的数值，例如用直线拟合样本点，再根据 x 预测 y。
3. 聚类：把相似的数据归成一组，例如按词向量的远近，把意思相近的词归到一起。
4. 降维：用更少的维度表示数据，例如用维度合适的词向量代替维度极高的 One-hot 向量。

3.6 机器学习与传统编程

机器学习与传统编程最大的区别，在于规则的来源：

	传统编程	机器学习
规则来源	人编写规则	数据提供规律
机器的工作	执行规则	学习参数
适合的问题	边界清楚的问题	规则难写的问题

符号主义的做法与传统编程一致，识别小狗与翻译这类规则难写的问题，适合交给机器学习。规则来源的这一差别，是 AI 能处理复杂问题的根本原因。

3.7 深度学习与网络结构

深度学习，英文是 Deep Learning，指层数较多的神经网络。深度指的是层数，层数越多，表示能力越强。输入层与输出层之间的隐藏层越多，网络就越深。

不同神经网络的区别在于结构，结构按任务设计：

网络结构	适合处理的数据
卷积神经网络 CNN	图像
循环神经网络 RNN	序列
长短期记忆网络 LSTM	长序列
Transformer	文本与多模态

LSTM 是对 RNN 的改进，适合更长的序列。多模态指文字，图像与语音等不同形式的信息。模型架构按任务设计，决定信息在网络中怎样流动。

小结

训练神经网络是在寻找一组合适的参数，损失函数衡量预测与真实答案的差距，优化方法负责逐步减小这个差距。模型复杂到只会记忆训练数据时就是过拟合，训练追求的是对新数据同样有效的泛化能力。机器学习分为训练与推理两个阶段，按训练数据是否带答案，又分为监督，无监督与半监督三种方式。深度学习使用多层神经网络，不同结构的网络各自适合图像，序列或文本这类不同的数据。

04
语言数字化

4.1 把语言写成预测问题

大语言模型建立在深度学习之上。要用神经网络处理语言，先要把语言任务改写成可以训练的预测问题，即把文字任务写成输入和输出。

以预测下一个词为例，输入是“今天天气真”，输出可以是“好”“热”或“冷”。以判断情感为例，输入是“这电影太棒了”，输出是“正向”。两个任务都符合 $y = f(x)$ 的形式，x 是一段文字，y 是要预测的结果。写成输入和输出之后，要解决的问题就变成：怎样让计算机理解文字。

4.2 文字的编码问题

神经网络的输入只能是数字，不能直接是文字。所以语言模型的第一步，是把词变成数。

把文字变成计算机能识别的数字，是一个编码问题。“他”“我”“开心”这几个词，都要分别编码成 `0101...` 这样的二进制数字。编码方式不同，会直接影响模型能否理解词与词之间的关系。编码方案有三种，前两种简单直接，第三种能表示词义。

4.3 编号与 One-hot 编码

第一种方案是给每个词一个编号，例如“他”记作 1，“我”记作 2，“开心”记作 3。这种做法简单，但编号不含语义。3 比 1 大，不代表“开心”与“他”之间有大小关系，词与词的关系也丢失了。编号只能表示词的身份，不能表示含义。

第二种方案是 **One-hot 编码**，每个词用一个很长的向量表示。向量是一串有顺序的数字，数字的个数叫维度。One-hot 向量的维度等于词表中词的个数，每个词只在自己对应的那一位上是 1，其余位置都是 0。这种编码能清楚地表示词的身份，实现也简单。缺点是维度极高，词表越大，向量越长。不同词的 1 落在不同的位置，任意两个向量互相正交，从向量上看不出两个词的远近，语义关系仍然没有表示出来。

4.4 词向量

更好的方案是 Word Embedding，即**词向量**：用维度合适的向量表示词义。例如“桌子”可以表示成 $(0.12, 0.00, 0.43, 0.50, \ldots)$ 这样一串小数。每个词变成一个稠密向量，长度既不太短，也不太长。稠密指大多数位置上都是非零的数值，不像 One-hot 向量那样几乎全是 0。与编号和 One-hot 编码相比，词向量中的数值携带了词义，意思相近的词，向量也相近。

4.5 余弦相似度与语义空间

词变成向量之后，词与词之间的关系就可以计算。常用的度量是余弦相似度：两个词向量 a 与 b 的相似度，等于两者夹角 θ 的余弦 $\cos\theta$。余弦相似度可以由点积算出：

$$\cos\theta = \frac{a \cdot b}{|a|\,|b|}$$

其中 $a \cdot b$ 是点积，即两个向量对应位置的数相乘再相加，$|a|$ 与 $|b|$ 是两个向量的长度。点积除以两个长度的乘积，结果只与方向有关。夹角越小，$\cos\theta$ 越接近 1，两个词的意思越相近。语义相近的词，在向量空间里的位置也更接近。

语义关系就这样被编码进了向量空间。图中把六个词的词向量画在平面上，空心点是桌子，椅子与沙发，实心点是键盘，鼠标与屏幕。意思相近的词聚在一起，两组词分别占据平面上不同的区域。

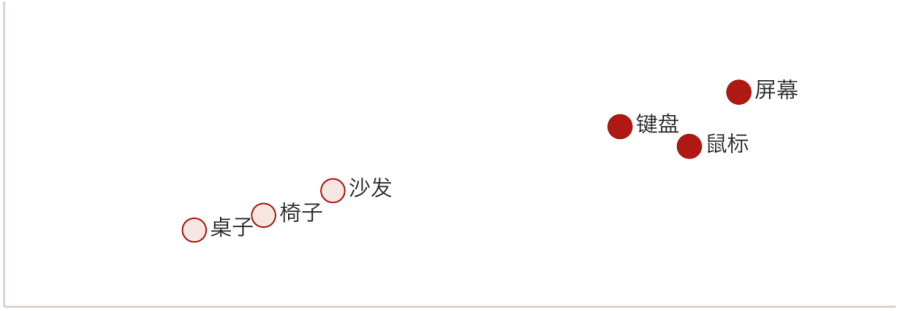


图 6　六个词的词向量：空心点桌子，椅子与沙发聚在左下，实心点键盘，鼠标与屏幕聚在右上。

4.6 嵌入矩阵与序列模型

所有词向量合起来，就是**嵌入矩阵**，英文是 Embedding Matrix。矩阵的每一行是一个词的向量，每一列是一个维度，d1，d2 与 d3 表示前三个维度：

词	d1	d2	d3
桌子	0.12	0.00	0.43
椅子	0.10	0.03	0.39
键盘	0.61	0.28	0.14

桌子与椅子的三个数值很接近，键盘与两者相差较大，这与语义相近的词在向量空间里更接近相一致。嵌入矩阵也是训练得到的模型组件，其中的数值与参数 w，b 一样，通过训练确定。

词向量可以作为神经网络的输入。词先经过 Embedding 变成词向量，词向量再依次送进序列模型，由序列模型给出输出结果。RNN 与 LSTM 曾经是处理序列的经典模型。词变成向量之后，自然语言就可以交给神经网络处理。

小结

神经网络只接受数字，语言任务必须先完成编码。编号与 One-hot 编码都只能区分词的身份，表达不了词义。词向量用维度适中的稠密向量承载词义，向量之间的相似程度可以用余弦相似度衡量，全部词向量组成嵌入矩阵。词向量作为输入送进序列模型，大语言模型处理的也是这样的向量。

05
大语言模型

5.1 LLM 与参数规模

大语言模型的英文是 Large Language Model，简称 LLM。LLM 可以概括为一个极大的，以语言预测为核心的神经网络。“极大”说的是规模，“以语言预测为核心”说的是模型要完成的任务。

LLM 叫“大”，原因是参数很多。参数就是训练神经网络时要找的 w 和 b，也是模型真正要学的东西。参数越多，模型能表达的关系通常越复杂，但成本也越高。

从这个角度看，LLM 仍然是用数据训练出来的函数，只是参数的数量大得多。

5.2 下一词预测与概率

大模型学习语言的核心任务是预测下一个词，英文叫 Next Token Prediction。以“今天天气真”为例，这几个字是前文。模型根据前文，对下一个位置可能出现的词分别计算概率，例如“好”“热”“冷”，再按概率给出下一个词，这里是“好”。前文，概率和下一个词，构成一次预测的三个环节。

这个任务很简单，但在海量文本上重复亿万次以后，模型就学到了大量语言规律。

因此，模型回答问题时，并不是在查找事实，而是在计算概率。模型生成的是“最像答案”的文字，不能把这种生成误认为思考。

理解幻觉和 RAG 都要用到这一点。模型缺少某方面的知识时，同样会生成一段看起来合理的文字，这段文字没有事实依据，就是幻觉。RAG 通过在回答前检索资料来减少幻觉。

5.3 Transformer 与注意力

Transformer 是大语言模型背后的网络结构，关键在于**注意力机制**，英文叫 Attention。模型读句子时，会自动判断哪些词更值得关注，并据此给不同的词分配不同的权重。英文说法“Attention lets tokens look at each other”讲的也是这件事：句子里的每个词都能看到其他词，再决定重点看谁。

模型读句子时看全句，找出词与词之间的关系，再给每个词分配权重，让重要的词得到更多关注。这种机制适合处理长文本和较长的上下文，训练时也可以并行计算。

有了注意力机制，模型不再平均对待句子里的每个词，而是能分清主次。

5.4 预训练与对齐

聊天模型要经过预训练与对齐两个阶段才能得到。

预训练的英文是 Pre-training，做法是把模型放进海量文本里学习语言规律。训练材料是互联网文本，训练任务是预测下一个词，模型在反复预测中学到语言规律，最后得到基座模型。基座模型好比一个知识很多的学霸，掌握了大量语言知识，但还不具备指令遵循与对话能力，不会按人的要求作答。

对齐的英文是 Alignment，作用是教模型按人类希望的方式回答。对齐用到问答数据和人类偏好，让模型学会遵循指令，最后得到聊天模型。对齐之后，模型才更接近日常使用的 ChatGPT 类产品。

阶段	训练材料	学到的能力	得到的模型
预训练	互联网文本	通过预测下一个词学习语言规律	基座模型
对齐	问答数据与人类偏好	指令遵循	聊天模型

预训练让模型掌握语言，对齐让模型学会按人的要求回答。

5.5 规模与训练成本

参数，数据和算力一起变大，模型的能力会发生跃迁，这种现象叫规模带来的涌现能力。量变积累到一定程度，模型就会出现复杂能力。推理，翻译和写代码等能力，都是在规模化训练中显现出来的。

规模扩大以后，收益与代价同时增长，训练成本变得很高。

收益	代价
能力更强	算力昂贵
泛化更好	数据昂贵
适配任务更多	工程复杂

从零训练大模型的投入很大，通常只有少数机构能承担。多数团队使用已经训练好的模型，在闭源模型与开源模型之间选择。

5.6 闭源模型与开源模型

闭源模型能力强，但核心不可控，典型的使用方式是通过 API 调用。开源模型可以部署和定制，但有门槛：权重可下载，部署工程要自己完成。这里的权重指模型训练好的参数。

类型	优点	缺点
闭源模型	效果稳定，接入简单，安全策略成熟	持续付费，数据有出域风险，难以深度定制
开源模型	成本可控，数据不出域，可以修改和部署	部署复杂，硬件要求高，效果需要调优

两者在数据处理上的差别很明显。调用闭源模型时，数据要交给第三方处理，存在出域风险。开源模型部署在自己的机器上，数据不出域。因此，开源模型适合对隐私和控制权要求高的场景。开源模型的成本优势体现在长期，前提是调用量足够大，而且要先投入硬件与运维。

5.7 按场景选模型

了解主流模型时，不能只比较模型的强弱，而要按场景选择。不同场景看重的能力不同。

场景	选择时看重
写代码	推理与工具调用能力，生成代码的质量和稳定性
长文档	上下文长度
企业落地	隐私与成本，部署方式与合规要求
多模态	能处理的输入输出形态，例如图像与语音

模型的优劣只能结合具体任务判断。

选模型时要同时权衡效果与成本，以及隐私与部署。追求效果且预算充足，可以直接调用 API。数据必须私有化，或者调用量大到长期付费超过自建投入，再考虑本地部署开源模型。本地部署要购置显卡等硬件，还要投入运维，这些都要算进成本。

小结

LLM 的“大”体现在参数数量上，核心任务是根据前文预测下一个词。模型按概率生成文字，并不查证事实，所以会产生幻觉。注意力机制让模型读句子时把更多权重放在重要的词上。预训练产出基座模型，对齐把基座模型变成聊天模型。多数团队不从零训练，而是按任务要求，在闭源模型的 API 和开源模型的本地部署之间选择。

06
微调与部署

6.1 从通才到专才

通用的大模型像一个通才，覆盖面广，但不专精。让通才模型变成专才模型的办法是微调。微调与本地部署是大模型落地的两条重要分岔路：微调让模型更懂某个领域，本地部署让模型运行在自己的机器上。

微调的英文是 Fine-tuning，作用是把通才培养成专才。微调不是从零训练，而是在已有模型上继续训练。基座模型通用能力强，但不熟悉业务要求的格式。微调后的模型更懂领域，更守格式，也更适合固定任务。

6.2 毛坯房与精装修

预训练像盖毛坯房，微调像做精装修，两者的成本差距很大。

对比项	预训练	微调
起点	从零开始	基于已有模型
数据	海量数据	垂直数据
算力成本	极高	低很多

垂直数据指某个具体领域或业务的数据。微调的核心是“小范围改变参数”，让模型更适合具体任务。毛坯房已经盖好，精装修只做局部改动，花费因此小得多。

6.3 微调的常见类型

按调整的方向，微调常见的有四类，其中 SFT 与领域微调用得最多。

类型	调整的方向
SFT	指令微调，让模型更好地按指令回答
领域微调	让模型更懂法律，医学和金融这类专业领域
格式微调	让模型按固定格式输出
风格微调	让模型的说法符合组织口径

微调会让模型“偏科”，也就是在目标领域和固定任务上表现更好。这种专门化符合业务需要。

6.4 微调的投入与归属

微调同样有成本，投入集中在三方面：高质量的标注数据，训练与评测所需的算力，以及持续的版本维护。所以微调适合需求明确，数据充分并且长期稳定的场景。

如果需求经常变化，RAG 往往比微调更灵活。RAG 让模型在回答前检索资料，资料变化时只需更新资料，不必重新训练模型。

微调之后模型的存放位置分两种情况。通过 API 微调，模型由厂商托管，留在云端。用开源模型微调，模型由自己持有，可以自己部署。所以，通过 API 微调并不等于模型掌握在自己手里。

6.5 本地部署的价值

本地部署的英文是 Local Deployment，指让模型运行在自己的机器上。本地部署之后，数据不出自己的服务器，调用也不依赖第三方。政府，金融和军工等场景尤其重视这一点。闭源模型以 API 调用为主，本地部署一般使用权重可下载的开源模型。

本地部署的好处可以归到安全，成本和控制权三方面。安全指数据安全，数据留在自己的服务器上。成本指长期成本可控，不必像调用 API 那样持续付费。控制权包括系统完全可控和可以接入内部系统两点，模型与服务都在自己手里，还能与组织内部的系统对接。

因此，本地部署适合调用量大，隐私要求高或者需要深度定制的组织。

6.6 硬件与显存估算

本地部署最大的门槛是硬件，因为参数要占用显存。**显存**是显卡自带的内存，模型运行时参数放在显存里。模型越大，参数越多，占用的显存越多，推理速度也越慢。硬件成本随之上升，大模型往往需要多卡服务器和 A100 或 H100 这类专业显卡，此外还有运维成本。所以本地部署要先算清楚购买显卡的预算。

粗略估算显存时，可以记住参数量与显存的量级关系。

模型规模	半精度显存	直观理解
7B	约 14GB	单张中高端显卡
13B	约 26GB	显存需求更高
70B	约 140GB	多张专业显卡

模型规模里的 B 表示十亿个参数，7B 就是七十亿个参数。半精度下每个参数约占 2 个字节，所以半精度显存的 GB 数约是模型规模数字的两倍。实际部署还会受量化，上下文长度和并发的影响。量化能减少显存占用，上下文越长，并发越多，显存占用越高。

显卡还分消费级和生产级，两者定位不同，不能只看显存数字。消费级显卡价格较低，适合学习和实验，但稳定性和扩展能力有限。生产级显卡显存更大，适合长时间运行，但成本高。学习阶段可以先从小模型和 API 入手。

6.7 本地部署的时机

本地部署不是越早越好，要按场景决策。

适合本地部署	不适合本地部署
数据安全要求高	刚起步的小团队
调用量巨大	预算有限
需要深度定制	需求还不稳定

多数项目在初期使用 API，投入更小。业务验证可行之后，再考虑本地部署。

小结

微调在已有模型上用垂直数据做小范围训练，让通用模型适应特定的领域，格式和口径。微调的数据，算力和维护投入适合稳定的需求，需求多变时 RAG 更合适。本地部署换来数据安全和控制权，代价是硬件与运维投入。估算显存时，半精度下每十亿个参数约占 2GB。是否本地部署取决于数据要求，调用量和预算，起步阶段先用 API。

07
RAG 工程

7.1 幻觉与检索

再强的模型也会产生幻觉，即在缺少相关知识时，仍然生成一段看起来合理但没有依据的回答。RAG 用检索资料的办法减少幻觉。

减少幻觉的关键是让模型在回答前查找资料，而不是靠重新训练解决所有问题。模型记不住的知识，可以在回答前先检索出来，交给模型参考。重新训练的代价太高，回答前检索资料更灵活，也更容易用工程手段实现。

7.2 检索增强生成

RAG 是 Retrieval-Augmented Generation 的缩写，中文叫检索增强生成，做法是先检索资料，再生成答案。三个英文词各对应一个环节。Retrieval 是检索，先查找资料。Augmented 是增强，用查到的资料补充模型的输入。Generation 是生成，由模型写出答案。

RAG 好比开卷考试，答题时不靠死记硬背，而是参考资料作答。对模型来说，回答的依据来自检索到的资料，而不只是参数里记住的知识。

RAG 的常见架构是应用夹在资料和模型之间。用户的问题不直接交给模型，而是先进入检索系统。检索系统到知识库里查找相关资料，查到的资料组成上下文，模型再根据上下文回答。

7.3 入库与查询

RAG 的工作分准备资料和回答问题两个阶段，分别叫文档入库 Pipeline 和查询 Pipeline。

准备资料时，文档先被切成小的文本块，文本块的英文是 **Chunk**。每个文本块经过向量化转成向量，向量化的英文是 Embedding。这些向量最后存进**向量数据库**。

回答问题时，用户的问题经过同样的向量化，系统在向量数据库里检索与问题相近的文本块，把这些文本块拼接成上下文，再交给模型生成答案。两个阶段各有一条流程。

- 准备资料：加载文档 → 切分文本块 → 向量化 → 存入向量数据库

- 回答问题：接收问题 → 问题向量化 → 相似检索 → 拼接上下文 → 生成答案

两个阶段能够衔接，是因为问题和资料在同一个向量空间里比较相似度。这里用到的仍是词向量的性质：语义相近，向量就更接近。

7.4 RAG 解决的问题

RAG 为大模型落地解决了四个关键问题。

改进	含义
知识可以更新	资料变化时只需更新知识库，不必重新训练模型
私有数据可以使用	组织内部的资料放进知识库，模型就能依据这些资料回答
幻觉明显减少	模型依据检索到的资料回答，不再凭空回答
答案可以引用	回答能指出依据来自哪份资料，便于追溯

RAG 不能彻底消除幻觉，但能显著降低出现幻觉的风险。

在 RAG 里，检索负责找依据，模型负责表达，相当于把“会说话”的模型接到“有资料”的系统上。

7.5 标准流程与向量数据库

标准 RAG 流程分五步：加载文档 → 切分文本块 → 向量入库 → 检索召回 → 模型生成答案。前三步属于准备资料阶段，后两步属于回答问题阶段。RAG 工程里的各种技术细节，都是在优化这五步。

向量数据库的英文是 Vector Database，是存放语义向量的数据库。语义相近的文本，向量之间的距离更近。因此，向量数据库可以按“意思相近”查找资料，而不只是按“关键词相同”查找。

7.6 检索环节的优化

模型依据检索到的资料回答，所以检索召回的质量决定模型能拿到什么资料。检索环节常用的优化有三种：QA 改写，BM25 关键词检索和重排序，分别作用在召回之前，召回之中和召回之后。

QA 改写的英文是 Query Rewriting，作用是先把问题改写成更适合检索的形式。用户的提问方式，不一定适合直接拿去检索。改写的过程是：先拿到口语化的问题，补全缺失的语境，生成适合检索的问题，再进入召回。

BM25 是关键词检索的经典方法。检索不能只依赖向量检索，关键词匹配有时比语义检索更稳定，专有名词和编号，以及法规条款和错误代码这类内容尤其如此。这类内容要求字面一致才能找准。

重排序的英文是 Rerank，作用是把召回结果重新排一遍。召回阶段先取出排在前面的一批结果，记作 TopK。然后对每条结果逐条打分，按分数重新排序，只保留排在前面的少数几条，记作 TopN。召回的目标是覆盖全面，尽量不漏掉相关资料。重排的目标是排序准确，把真正相关的资料排在前面。

7.7 Chunk 与切块策略

Chunk 是 RAG 里的最小知识片段。长文档必须切成小块，才能入库和检索。Chunk 太大，检索不准确。Chunk 太小，语义不完整。所以切块策略直接决定 RAG 效果的上限。

切块可以按长度，按标题或者按语义进行，没有一种切法适合所有文档。

切分方式	做法	特点
简单切分	按固定长度切，加 overlap	实现容易
结构切分	按标题和段落切	保留层级，更适合知识库
语义切分	按意思切，在话题转换处切开	每块意思完整，实现更复杂

overlap 指相邻两块之间重叠的一段内容，作用是避免一句话被切断后意思丢失。

合理的切块不是把文档切碎，而是保留能被检索的意义单元。

7.8 文件处理与 Markdown

文件处理是 RAG 工程的地基。文件先经过解析，再做清洗，然后切块，最后入库。这一步遵循“Garbage in, garbage out”的原则，即输入的质量决定输出的质量。文档处理不干净，后面的向量库和大模型都会受到影响。

Markdown 很适合做知识库的原材料，因为结构清楚，层级天然存在。一份 Markdown 文档以标题统领内容，标题之下是段落，列表和表格。与杂乱的 Word 和 PDF 文件相比，Markdown 更容易被稳定解析。

Markdown 的常见语法只有五类，这些结构信息能帮助 RAG 更好地切分和检索。

写法	表示的结构
# 开头的行	标题
- 开头的行	列表
> 开头的行	引用
三个反引号包住的内容	代码块
用竖线分隔的行	表格

整理资料可以借助常见的 Markdown 工具，例如 Typora 和 Obsidian，以及语雀和飞书文档。工具不是重点，重点是把知识整理成干净的文本，输出结构清楚，便于解析的文档。

7.9 docx 与 PDF 的处理

处理 docx 文档不能只提取正文。一份 docx 里要提取的内容有正文与标题层级，以及表格与图片说明。如果表格和图片丢失，知识库就会缺少关键信息。

PDF 要先分清是文本型还是图片型，两种 PDF 的处理方法完全不同。

文本型 PDF	图片型 PDF
文字可复制	每页都是图片
可以直接解析	需要 OCR
结构可能混乱	识别质量决定效果

区分的办法是看文字能不能复制。文本型 PDF 可以直接解析，但解析出的结构可能混乱，还需要清洗。图片型 PDF 由页面图片组成，必须先用 OCR 把文字识别出来，识别质量决定后续效果。很多扫描版资料属于图片型 PDF，需要先经过 OCR。

7.10 OCR 识别与文件入库

OCR 是 Optical Character Recognition 的缩写，OCR 模型的作用是把图片里的文字识别出来。识别过程分四步：输入扫描图片，先做版面分析，再做文字识别，最后做结构还原。文字识别负责认出字，版面分析和结构还原负责尽量保留段落，表格和阅读顺序，所以 OCR 的工作不只是识字。

一份图片 PDF 要变成可以检索的文本，需要经过四个步骤。

1. 上传文件。
2. 调用 OCR 模型识别文字。
3. 把识别结果转成 Markdown。
4. 把 Markdown 文件送进 RAG 的知识库。

经过这四步，原始文件就成了知识库里可以检索和问答的内容。

小结

RAG 让模型先检索资料再生成答案，回答因此有了依据，幻觉随之减少。系统分两个阶段工作：准备阶段把文档切块，向量化后存入向量数据库，回答阶段按语义相似度找出相关文本块交给模型。检索质量可以从三处提升：召回前改写问题，召回时结合关键词检索，召回后重新排序。切块与文件处理决定效果上限，知识库的原材料适合整理成 Markdown，图片型 PDF 要先做 OCR。

08
Agent 与总结

8.1 智能体的组成与分类

智能体的英文是 Agent，作用是让模型不只回答问题，还能完成任务。智能体的组成可以写成一个式子：“Agent = 大模型 + 工具 + 记忆 + 规划”。

四个组成部分各有分工。大模型负责理解含义和组织表达。工具让智能体能调用外部能力，例如搜索和计算。记忆让智能体记住之前的对话和中间结果。规划让智能体把复杂任务拆成步骤，按顺序完成。

智能体可以按能力和形态分成四类，从单纯的对话一直延伸到完整的工作流。

类型	特点
对话型	以对话为主
工具型	能调用工具完成任务
工作流型	按设定的流程分步执行
多智能体	多个智能体分工协作

分类只是名称上的区别，衡量智能体的标准是能否稳定地调用工具完成任务。

8.2 智能体开发框架

智能体开发框架用来组织复杂流程。框架管理的对象包括模型，工具和记忆，也包括规划，工作流和监控。框架同时管理运行时的状态和流程，把一次性的聊天变成可以重复运行的工程系统。

agno 是一个用来创建智能体的框架，借助框架可以降低组织流程的复杂度。agno 的基本用法是：定义模型，工具和指令，再让 Agent 执行任务。学习 agno 时，重点看 agno 如何把 LLM，工具和 RAG 连接起来。

8.3 agno 智能体的结构

用 agno 创建智能体，从最小的结构开始。

```
agent = Agent(
  model=llm,
  tools=[search, calculator],
  instructions="按资料回答问题"
)
```

这段代码是一个骨架，只展示智能体的结构，不能单独运行。代码创建了智能体 `agent`，三个参数对应定义智能体的三样东西。

参数	含义	骨架里的写法
`model`	模型	`llm`
`tools`	工具列表	`search` 和 `calculator`
`instructions`	指令	`"按资料回答问题"`

`llm`，`search` 和 `calculator` 代表需要事先准备好的模型和工具。搭建时先让 Agent 能听懂指令，再逐步增加工具能力。

让智能体带着资料回答，要在这个结构上接入检索器和知识库。检索器的英文是 Retriever，承担 RAG 架构里检索系统的角色。知识库的英文是 Knowledge Base。一次回答的流程是：

用户问题 → Agent → Retriever → 知识库 → 带依据的回答

Agent 接到问题后，通过 Retriever 到知识库里查找资料，再依据查到的资料回答。骨架里没有给出检索器和知识库的写法，接入时要按 agno 框架的接口补全。

8.4 从函数到智能体

全天内容沿着一条路线展开，从函数开始，到智能体结束。

阶段	核心概念
函数	输入 x 经过规则 f 得到输出 y
机器学习	从数据中学习函数
神经网络	多输入，多参数和多层组合，用来逼近复杂函数
词向量	每个词变成一个稠密向量，语义相近的词更接近
大语言模型	参数极多，以预测下一个词为核心的神经网络
RAG	先检索资料，再生成答案
智能体	大模型加上工具，记忆和规划

AI 落地要把模型能力接到真实数据和真实工具上。RAG 负责接入真实数据，智能体负责接入真实工具。

小结

智能体在大模型之外增加了工具，记忆和规划，可以调用工具完成任务。开发框架负责管理智能体的组成部分和运行流程。agno 用模型，工具和指令三个参数定义智能体，接入检索器和知识库后，智能体就能依据资料回答。RAG 与智能体分别让模型用上数据和工具，构成 AI 落地的两条路径。

练习与作业

复习题

1. 写出人工智能定义中的四个环节，以及全天学习路径依次经过的六个概念。
2. 比较符号主义与联结主义，说明两条路线的规则分别来自哪里，并用识别小狗的例子说明符号主义的局限。
3. 写出 $f(x) = g(wx + b)$ 中线性部分，激活函数和非线性输出各对应哪一部分，并说明激活函数的作用。
4. 说明什么样的参数才算好，写出求参数时从随机参数到损失变小的四个步骤，并比较过拟合与泛化。
5. 说明设计语言模型时要先解决的问题，比较每个词一个编号，One-hot 编码与词向量三种编码方式的优缺点。
6. 说明 LLM 名字里“大”的含义，写出大模型学习语言的核心任务，并说明模型按概率生成文字与幻觉之间的关系。
7. 比较预训练与对齐用到的材料和得到的模型，并写出闭源模型与开源模型各自的优点和缺点。
8. 说明通才模型变成专才模型的途径，写出微调常见的四个类型，并说明需求经常变化时 RAG 比微调更灵活的原因。
9. 按“7B 约 14GB”的量级，写出 13B 和 70B 模型半精度部署大约需要的显存，并写出适合与不适合本地部署的场景各三条。
10. 写出 RAG 准备资料与回答问题两个阶段的流程，并说明 QA 改写，BM25 和重排序分别作用在召回的哪个位置。
11. 说明 Chunk 太大和太小各会造成的问题，区分文本型 PDF 与图片型 PDF 的处理方式。
12. 写出智能体的四个组成部分，说明用 agno 定义智能体的三个参数，以及 Retriever 在带资料回答时的作用。

课后作业

课后作业有两个练习，分别对应 OCR 文件处理和 agno 智能体两部分内容。

练习一 OCR 文件处理

目标是调用 OCR 模型处理一份图片 PDF，例如扫描版资料，得到可以检索的文本，让这份原始文件成为知识库里可以问答的内容。

1. 准备一份图片型 PDF，上传给 OCR 模型。
2. 运行识别，检查识别结果是否保留了段落，表格和阅读顺序。
3. 将识别结果保存为 Markdown 文件。

4. 把这个 Markdown 文件加入 RAG 的知识库。

完成的标志是针对这份文件的内容提问，RAG 能依据文件内容给出回答。

练习二 agno 智能体与 RAG

目标是用 agno 创建一个能依据资料回答问题的智能体。代码骨架只给出了结构，模型与工具，以及检索器与知识库，都要按 agno 框架的接口补全。

1. 参照 agno 智能体结构的代码骨架创建智能体，定义模型，工具和指令。
2. 先让 Agent 能听懂指令，再逐步增加工具能力。
3. 为智能体接入 Retriever 和知识库，知识库可以使用练习一得到的 Markdown 文件。
4. 向智能体提问，检查回答是否依据知识库里的资料。

完成的标志是智能体能带着知识库里的依据回答问题。